

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

Bộ môn: Công nghệ thông tin



BÀI TẬP LỚN MÔN HỌC
KỸ THUẬT LẬP TRÌNH NÂNG CAO

NGÀNH : KỸ THUẬT MÁY TÍNH

HỆ : ĐẠI HỌC CHÍNH QUY

Sinh viên : Trần Thị Thu Hà

MSSV : K225480106009

Lớp : K58KTP.01

Giáo viên hướng dẫn : TS. Nghiêm Văn Tính

Thái Nguyên – 2026

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

Bộ môn: Công nghệ thông tin



BÀI TẬP LỚN MÔN HỌC
KỸ THUẬT LẬP TRÌNH NÂNG CAO

ĐỀ TÀI: THIẾT KẾ CHƯƠNG TRÌNH MÃ HÓA – GIẢI MÃ
CHUỖI KÝ TỰ

Sinh viên : Trần Thị Thu Hà

MSSV : K225480106009

Lớp : K58KTP.01

Giáo viên hướng dẫn : TS. Nghiêm Văn Tính

Thái Nguyên – 2026

BÀI TẬP LỚN

MÔN HỌC: KỸ THUẬT LẬP TRÌNH NÂNG CAO

Sinh viên : Trần Thị Thu Hà

MSSV : K225480106009

Lớp : K58KTP

Ngành : Kỹ thuật máy tính

Bộ môn: Công Nghệ Thông Tin

Giáo viên hướng dẫn: TS. Nghiêm Văn Tính

Ngày giao đề :

Ngày hoàn thành :

Tên đề tài : Thiết kế chương trình mã hóa – giải mã chuỗi ký tự.

Yêu cầu :

- Xây dựng chương trình mã hóa – giải mã chuỗi ký tự
- Thiết kế giao diện

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

Xếp loại:

Điểm:

Thái Nguyên, ngày tháng năm 2025.

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

MỤC LỤC

LỜI CAM ĐOAN.....	I
LỜI CẢM ƠN.....	II
LỜI NÓI ĐẦU	III
DANH MỤC HÌNH ẢNH	IV
CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI.....	9
1.1. Giới thiệu đề tài.....	9
1.2. Mục tiêu đề tài	10
1.3. Phạm vi đề tài	12
1.4. Ý nghĩa của đề tài.....	13
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	15
2.1. Khái niệm mã hóa	15
2.1.1. Thuật toán mã hóa Ceasar Cipher	15
2.1.2. Thuật toán mã hóa Vigenere.....	17
2.1.3. Thuật toán mã hóa hiện đại (AES).....	17
2.2. Quy trình logic của hệ thống	18
CHƯƠNG 3. THIẾT KẾ CHƯƠNG TRÌNH.....	19
3.1. Mục tiêu thiết kế.....	19
3.2. Kiến trúc hệ thống.....	19
3.2.1. Tầng giao diện (Presentation Layer)	19
3.2.2. Tầng điều phối và kiểm soát (Controller Layer).....	20
3.2.3. Tầng xử lý thuật toán (Algorithm Layer)	20
3.2.4. Tầng lưu trữ và hỗ trợ (Storage và Support)	21
3.3. Sơ đồ khối các thuật toán chính	22
3.3.1. Thuật toán mã hóa Caesar	22
3.3.2. Thuật toán Vigenere	23
3.3.3. Thuật toán AES (Advanced Encryption Standard).....	24
3.3.4. Quy trình mã hóa và giải mã file.....	26
3.4. Cấu trúc dữ liệu	26
3.5. Các hàm chính trong chương trình	27

3.5.1. Hàm kiểm tra khóa <code>validate_key()</code>	27
3.5.2. Hàm mã hóa và giải mã của mã hóa Ceasar	27
3.5.3. Hàm mã hóa và giải mã của mã hóa Vigenere	28
3.5.4. Hàm mã hóa và giải mã của mã hóa AES	29
3.6. Phân tích dữ liệu đầu vào	29
3.6.1. Chuỗi văn bản (Plaintext/Ciphertext).....	29
3.6.2. Khóa (Secret Key)	29
3.7. Thiết kế hàm kiểm tra khóa (Key validation).....	30
3.8. Thiết kế xử lý file.....	30
3.9. Thiết kế GUI (giao diện người dùng).....	31
3.10. Thiết kế ghi Log (nhật ký hệ thống).....	31
CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG	32
4.1. Kết quả kiểm thử.....	32
4.2. Kết luận.....	34
4.2.1. Kết quả sản phẩm đạt được.....	34
4.2.2. Kiến thức đã học được trong quá trình làm đề tài	34
TÀI LIỆU THAM KHẢO.....	36
LINK GITHUB.....	37

LỜI CAM ĐOAN

Em xin cam đoan rằng đề tài ‘Thiết kế chương trình mã hóa – giải mã chuỗi ký tự’ là kết quả của quá trình nghiên cứu và thực hiện của em dưới sự hướng dẫn của thầy Nghiêm Văn Tính.

Toàn bộ nội dung trong đề tài này do em tự thực hiện dựa trên các kiến thức đã học và các nguồn tài liệu tham khảo được liệt kê trong danh mục tài liệu tham khảo. Em không sao chép nội dung của bất kỳ cá nhân hoặc tổ chức nào một cách trái phép. Nếu phát hiện có bất kỳ hành vi gian lận hoặc vi phạm về quy định nghiên cứu khoa học, em xin hoàn toàn chịu trách nhiệm trước nhà trường và pháp luật.

Em cam kết mọi thông tin, số liệu và kết quả trong đề tài này là chính xác và trung thực.

Thái nguyên, ngày 13 tháng 03 năm 2026

NGƯỜI CAM ĐOAN

(Ký và ghi rõ họ tên)

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc đến cô giáo hướng dẫn TS. Nghiêm Văn Tính đã tận tình giảng dạy, truyền đạt những kiến thức bổ ích và định hướng quý báu cho chúng em trong suốt quá trình học tập. Chính những nền tảng lý thuyết và kinh nghiệm thực tiễn mà cô đã chia sẻ là nguồn động lực quan trọng để em có thể hoàn thành bài tập lớn này.

Em cũng xin gửi lời cảm ơn đến các bạn trong lớp đã đồng hành, trao đổi, chia sẻ tài liệu và hỗ trợ em trong quá trình tìm hiểu, phân tích và triển khai nội dung. Mặc dù đã có nhiều cố gắng, nhưng do kiến thức và kinh nghiệm còn hạn chế, bài làm chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp của thầy cô và các bạn để có thể hoàn thiện hơn trong các bài sau này.

Một lần nữa, em xin chân thành cảm ơn!

LỜI NÓI ĐẦU

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ, dữ liệu cá nhân và thông tin tổ chức ngày càng trở thành mục tiêu của các cuộc tấn công mạng. Bảo mật thông tin không còn là vấn đề của riêng các chuyên gia an ninh mạng mà đã trở thành yêu cầu thiết yếu đối với bất kỳ lập trình viên nào. Xuất phát từ thực tế đó, đề tài "*Thiết kế chương trình mã hóa – giải mã chuỗi ký tự*" được lựa chọn thực hiện nhằm nghiên cứu và ứng dụng các kỹ thuật bảo mật cơ bản vào một sản phẩm phần mềm hoàn chỉnh.

Đề tài tập trung vào việc triển khai ba thuật toán mã hóa tiêu biểu cho ba thể hệ kỹ thuật khác nhau: **Caesar Cipher** – thuật toán cổ điển dựa trên dịch chuyển ký tự, minh họa rõ ràng khái niệm mã hóa thay thế; **Vigenère Cipher** – bước tiến hóa sử dụng từ khóa để tạo ra sự biến thiên trong mã hóa; và **AES (Advanced Encryption Standard)** – chuẩn mã hóa đối xứng hiện đại được áp dụng rộng rãi trong công nghiệp. Sự kết hợp này giúp người đọc thấy được sự phát triển của mật mã học qua từng giai đoạn lịch sử.

Chương trình được xây dựng bằng ngôn ngữ **Python**, sử dụng thư viện **Tkinter** cho giao diện đồ họa và **PyCryptodome** cho xử lý mã hóa AES. Ngoài chức năng mã hóa và giải mã văn bản trực tiếp, chương trình còn hỗ trợ xử lý file, ghi nhật ký hoạt động (log) và kiểm soát tính hợp lệ của dữ liệu đầu vào – hướng đến một công cụ có tính ứng dụng thực tế, không chỉ dừng lại ở mức minh họa lý thuyết.

Trong quá trình thực hiện, mặc dù đã cố gắng nghiên cứu và xây dựng chương trình một cách cẩn thận, song do kiến thức và kinh nghiệm còn hạn chế, báo cáo này chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý và nhận xét từ thầy TS. Nghiêm Văn Tính để có thể hoàn thiện hơn trong những đề tài tiếp theo.

DANH MỤC HÌNH ẢNH

Hình 3.1. Thuật toán mã hóa và giải mã Ceasar

Hình 3.2. Thuật toán mã hóa và giải mã Vigenere

Hình 3.3. Thuật toán mã hóa và giải mã AES

Hình 3.4. Mã hóa và giải mã file

Hình 4.1. Kết quả kiểm thử mã hóa Ceasar

Hình 4.2. Kết quả kiểm thử thuật toán giải mã Ceasar

Hình 4.3. Kết quả kiểm thử thuật toán Vigenere

Hình 4.4. Kết quả kiểm thử thuật toán AES

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1. Giới thiệu đề tài

Trong kỷ nguyên số hóa hiện nay, dữ liệu đã trở thành tài sản quý giá nhất của cá nhân và tổ chức. Sự bùng nổ của Internet và các thiết bị kết nối toàn cầu mang lại sự tiện lợi nhưng cũng tiềm ẩn nhiều rủi ro về an ninh mạng. Do đó, **bảo mật dữ liệu** không còn là lựa chọn mà là yêu cầu bắt buộc để chống lại các hành vi xâm nhập, đánh cắp thông tin trái phép.

Mã hóa dữ liệu (Encryption) đóng vai trò là "chốt chặn" cuối cùng. Bằng cách biến đổi thông tin từ dạng văn bản thuần túy (Plaintext) sang dạng bản mã (Ciphertext) không thể đọc hiểu nếu không có khóa, chúng ta đảm bảo được tính toàn vẹn và bảo mật của thông tin.

Các ứng dụng thực tế tiêu biểu gồm có :

- Bảo mật tin nhắn : Các ứng dụng như Zalo, Telegram sử dụng mã hóa đầu cuối (End-to-End Encryption).
- Quản lý mật khẩu : Lưu trữ mật khẩu người dùng dưới dạng băm hoặc mã hóa để tránh lộ lọt dữ liệu.
- Trong truyền tải dữ liệu : Có giao thức HTTPS giúp bảo vệ thông tin khi truyền đi trên môi trường web.
- Hệ thống ngân hàng : Bảo mật các giao dịch tài chính thông qua sổ dư tài khoản.

Đề tài này sẽ tập trung vào việc nghiên cứu và xây dựng chương trình mã hóa và giải mã mã chuỗi ký tự nhằm minh họa cách thức hoạt động cơ bản của các thuật toán bảo mật

1.2. Mục tiêu đề tài

Đề tài “*Thiết kế chương trình mã hóa – giải mã chuỗi ký tự*” được thực hiện nhằm đạt được các mục tiêu cụ thể sau:

a. Nghiên cứu cơ sở lý thuyết về mã hóa dữ liệu

Đề tài tập trung tìm hiểu các khái niệm nền tảng trong lĩnh vực bảo mật thông tin và mã hóa dữ liệu. Tiến hành nghiên cứu nguyên lý hoạt động của các thuật toán mã hóa cơ bản như Ceasar Cipher, Vigenere Cipher và một số phương pháp mã hóa hiện đại ở mức tổng quan.

Việc nghiên cứu này giúp làm rõ các thành phần chính của một hệ mã hóa, bao gồm: dữ liệu gốc (plaintext), khóa (key), thuật toán mã hóa (encryption algorithm), bản mã (ciphertext) và quá trình giải mã (decryption).

Thông qua đó, đề tài nhằm giúp em hiểu được cách dữ liệu được chuyển đổi từ dạng có thể đọc được sang dạng được bảo vệ và ngược lại, đồng thời nhận thức được vai trò của mã hóa trong việc đảm bảo tính bảo mật và toàn vẹn thông tin trong các hệ thống phần mềm hiện nay.

b. Xây dựng chương trình mã hóa chuỗi ký tự

Mục tiêu tiếp theo của đề tài là thiết kế và xây dựng chương trình có khả năng thực hiện mã hóa chuỗi ký tự đầu vào. Chương trình cho phép người dùng nhập một chuỗi văn bản bất kỳ và một khóa mã hóa tương ứng. Sau đó, hệ thống sẽ áp dụng thuật toán mã hóa đã lựa chọn để chuyển đổi chuỗi ban đầu thành bản mã.

Chương trình phải đảm bảo :

- Xử lý đúng các loại ký tự như chữ hoa, chữ thường, chữ số và ký tự đặc biệt
- Bảo toàn định dạng chuỗi ban đầu (không làm mất dữ liệu)

- Cho phép thay đổi khóa mã linh hoạt
- Hoạt động ổn định với các chuỗi có độ dài khác nhau

Việc xây dựng chức năng mã hóa giúp minh họa rõ ràng cách một thuật toán mã hóa hoạt động trong thực tế và cách triển khai chúng bằng ngôn ngữ lập trình.

c. Xây dựng chương trình giải mã dữ liệu

Bên cạnh chức năng mã hóa, đề tài đặt mục tiêu thiết kế quy trình giải mã nhằm khôi phục dữ liệu ban đầu từ bản mã. Chức năng giải mã phải đảm bảo rằng với cùng một khóa đã sử dụng trong quá trình mã hóa, chương trình có thể khôi phục lại chính xác chuỗi ban đầu mà không xảy ra sai lệch.

Việc xây dựng chức năng giải mã giúp kiểm chứng tính đúng đắn của thuật toán và đảm bảo tính toàn vẹn dữ liệu. Đồng thời, quá trình này cũng giúp người học hiểu rõ mối quan hệ giữa mã hóa và giải mã, cũng như cách hai quá trình này phụ thuộc vào cùng một khóa bảo mật.

d. Thử nghiệm và đánh giá chương trình

Sau khi hoàn thành việc xây dựng chương trình, đề tài tiến hành thử nghiệm trên nhiều trường hợp dữ liệu khác nhau để đánh giá độ ổn định và độ chính xác của hệ thống. Các trường hợp kiểm thử bao gồm:

- Chuỗi chỉ gồm chữ cái
- Chuỗi gồm chữ hoa và chữ thường
- Chuỗi chứa số và ký tự đặc biệt
- Chuỗi có độ dài lớn
- Chuỗi rỗng hoặc dữ liệu đặc biệt

Kết quả thử nghiệm sẽ được phân tích nhằm đánh giá :

- Tính chính xác của quá trình mã hóa và giải mã

- Khả năng xử lý các trường hợp biên
- Độ ổn định của chương trình khi chạy nhiều lần
- Khả năng mở rộng để tích hợp thêm các thuật toán mã hóa khác

Thông qua quá trình thử nghiệm và đánh giá, đề tài hướng tới việc hoàn thiện chương trình, đảm bảo tính đúng đắn, dễ sử dụng và có thể mở rộng trong tương lai.

e. Mục tiêu tổng quát

Tổng thể, đề tài hướng đến việc giúp nắm vững kiến thức về mã hóa dữ liệu, rèn luyện kỹ năng lập trình xử lý chuỗi, xây dựng thuật toán và kiểm thử phần mềm. Đồng thời, chương trình được xây dựng có thể làm nền tảng để phát triển các hệ thống bảo mật nâng cao hơn như mã hóa tệp tin, bảo mật mật khẩu hoặc tích hợp vào các ứng dụng thực tế.

1.3. Phạm vi đề tài

Đề tài tập trung vào các nội dung trọng tâm sau :

Chương trình thực hiện các chức năng chính :

- **Xử lý chuỗi văn bản trực tiếp:** Người dùng có thể nhập liệu, tùy chọn thuật toán, nhập khóa và nhận kết quả ngay trên giao diện đồ họa.
- **Mã hóa và giải mã File:** Hỗ trợ đọc các tệp tin văn bản định dạng .txt, tiến hành mã hóa và xuất ra tệp tin định dạng riêng .enc..
- **Quản lý Log (Nhật ký):** Mọi thao tác quan trọng của người dùng đều được hệ thống ghi lại vào tệp log.txt kèm theo mốc thời gian (Timestamp) để phục vụ việc kiểm soát và theo dõi.

- **Kiểm soát dữ liệu đầu vào (Validation):** Tự động kiểm tra tính hợp lệ của khóa (ví dụ: khóa Caesar phải là số, khóa Vigenere phải là chữ) trước khi thực hiện thuật toán để tránh lỗi hệ thống.

Các thuật toán được triển khai :

- **Mã hóa Caesar :** Thực hiện dịch chuyển ký tự dựa trên mã ASCII, tập trung vào bảng chữ cái Latinh.
- **Mã hóa Vigenere :** Mở rộng sang hệ mật đa bảng chữ cái, sử dụng từ khóa (keyword) để tăng cường độ phức tạp so với Caesar.
- **Mã hóa AES (Advanced Encryption Standard) :** Cài đặt chuẩn mã hóa hiện đại nhất hiện nay với chế độ **EAX**, kết hợp cùng hàm băm **SHA-256** để xử lý khóa và mã hóa **Base64** để tối ưu hóa việc lưu trữ chuỗi ký tự.

Môi trường và công cụ phát triển :

- **Ngôn ngữ lập trình:** Python
- **Thư viện giao diện:** Tkinter (thư viện chuẩn của Python).
- **Thư viện mật mã:** PyCryptodome (để xử lý thuật toán AES chuyên sâu).

1.4. Ý nghĩa của đề tài

Việc thực hiện đề tài không chỉ có ý nghĩa về mặt sản phẩm mà còn mang lại giá trị học thuật cao cho sinh viên:

- Về kiến thức : Giúp sinh viên nắm vững nền tảng về mật mã học (Cryptography) và cách thức bảo vệ dữ liệu.
- Về kỹ năng lập trình : củng cố kỹ năng xử lý chuỗi (String manipulation), tư duy logic và cấu trúc dữ liệu trong ngôn ngữ Python.

- Tính ứng dụng : Hiểu được cách áp dụng lập trình vào việc giải quyết các bài toán an ninh thông tin thực tế, tạo tiền đề cho việc nghiên cứu các hệ thống bảo mật phức tạp hơn.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm mã hóa

Mã hóa (Encryption) là một thành phần cốt lõi của mật mã học, đóng vai trò chuyển đổi dữ liệu từ dạng văn bản thuần túy, dễ hiểu sang một định dạng bị xáo trộn, không thể đọc được nếu không có công cụ hỗ trợ.

Quy trình cơ bản của mã hóa gồm hai giai đoạn đối nghịch :

- Quá trình mã hóa (Encryption) : Biến đổi Plaintext (Văn bản gốc) thành Ciphertext (Bản mã) thông qua một thuật toán và một khóa bí mật.
- Quá trình giải mã (Decryption) : Chuyển đổi ngược từ Ciphertext về lại Plaintext bằng cách sử dụng khóa tương ứng.

2.1.1. Thuật toán mã hóa Ceasar Cipher

Mật mã Caesar (còn gọi là mật mã dịch chuyển) là một trong những kỹ thuật mã hóa thay thế đơn giản nhất. Mỗi chữ cái trong văn bản gốc sẽ được thay thế bằng một chữ cái khác nằm cách nó một khoảng k vị trí trong bảng chữ cái.

Ví dụ với $k = 3$:

- Chữ A dịch sang phải 3 vị trí thành chữ D
- Chữ B dịch sang phải 3 vị trí thành chữ E
- Chữ X dịch sang phải 3 vị trí sẽ xoay vòng lại thành chữ A

Mô hình toán học : Để máy tính có thể hiểu và thực hiện, ta quy ước bảng chữ cái Latinh (26 chữ cái) thành các số từ 0 đến 25 ($A=0, B=1, \dots, Z=25$).

Công thức tổng quát được xác định như sau :

- Công thức mã hóa : $E(x) = (x + k) \pmod{26}$
- Công thức giải mã : $D(x) = (x - k) \pmod{26}$

Trong đó :

- x : Vị trí ký tự trong bảng chữ cái
- k : Khóa (số bước dịch chuyển).
- (mode 26) : Phép toán chia lấy dư để đảm bảo kết quả luôn nằm trong phạm vi 26 chữ cái.

❖ Ưu điểm của Ceasar Cipher

Dù là một thuật toán cổ điển, Caesar Cipher vẫn mang lại những giá trị nhất định trong học tập và ứng dụng cơ bản:

- **Độ phức tạp thấp:** Cấu trúc thuật toán cực kỳ đơn giản, dễ dàng cài đặt trên bất kỳ ngôn ngữ lập trình nào (C++, Python, Java...).
- **Tính minh họa cao:** Giúp người mới bắt đầu dễ dàng hình dung về khái niệm "Dịch chuyển" và "Khóa đối xứng".
- **Tốc độ xử lý:** Vì chỉ sử dụng các phép toán cộng trừ cơ bản, chương trình thực thi mã hóa và giải mã gần như tức thời.

❖ Nhược điểm và hạn chế

Do ra đời từ rất sớm, Caesar Cipher bộc lộ nhiều lỗ hổng nghiêm trọng trước các kỹ thuật tấn công hiện đại:

- **Bảo mật cực thấp:** Vì chỉ có 25 khả năng dịch chuyển (với bảng chữ cái tiếng Anh), kẻ tấn công có thể sử dụng phương pháp **Brute-force** (thử sai toàn bộ) chỉ trong vài giây để tìm ra bản gốc.

- **Dễ bị tấn công phân tích tần suất:** Trong mỗi ngôn ngữ, các chữ cái có tần suất xuất hiện khác nhau (ví dụ tiếng Anh chữ 'E' xuất hiện nhiều nhất). Kẻ tấn công có thể dựa vào tần suất của các ký tự trong bản mã để đoán ra khóa k.
- **Không bảo mật được dữ liệu phức tạp:** Thuật toán này chỉ hoạt động hiệu quả trên văn bản, không phù hợp cho các định dạng dữ liệu hiện đại như hình ảnh, video hay file thực thi.

2.1.2. Thuật toán mã hóa Vigenere

Vigenere cải tiến từ Caesar bằng cách sử dụng một từ khóa (Keyword). Mỗi chữ cái trong từ khóa sẽ xác định một khoảng dịch chuyển khác nhau cho từng ký tự tương ứng trong văn bản gốc.

- **Nguyên lý:** Nếu từ khóa ngắn hơn văn bản, từ khóa sẽ được lặp lại tuần hoàn.
- **Ưu điểm:** Khắc phục được nhược điểm của Caesar bằng cách làm xáo trộn tần suất xuất hiện của các chữ cái, giúp chống lại phương pháp phân tích tần suất cơ bản.

2.1.3. Thuật toán mã hóa hiện đại (AES)

AES (Advanced Encryption Standard) là chuẩn mã hóa đối xứng hiện đại, được chính phủ Hoa Kỳ phê duyệt. Khác với các thuật toán cổ điển xử lý trên bảng chữ cái, AES xử lý dữ liệu trên các khối (blocks) nhị phân

❖ Chế độ vận hành EAX (Authenticated Encryption)

Trong chương trình, AES được triển khai dưới chế độ **EAX**. Đây là chế độ mã hóa xác thực (AEAD) giúp đảm bảo đồng thời:

- **Tính bảo mật (Confidentiality):** Không ai có thể đọc nội dung nếu không có khóa.
- **Tính toàn vẹn (Integrity):** Phát hiện ngay nếu bản mã bị chỉnh sửa trái phép thông qua thẻ xác thực (**Tag**).

❖ **Cấu trúc khóa và dữ liệu**

- **Kích thước khối:** 128 bit.
- **Nonce (Number used once):** Một số ngẫu nhiên duy nhất được tạo ra cho mỗi lần mã hóa để đảm bảo cùng một văn bản gốc mã hóa hai lần sẽ ra hai bản mã hoàn toàn khác nhau.

2.2. Quy trình logic của hệ thống

Hệ thống vận hành theo một luồng xử lý chặt chẽ:

Bước 1: Tiếp nhận: Người dùng nhập văn bản và khóa

Bước 2: Kiểm tra (Validation): Xác định định dạng khóa phù hợp với thuật toán đã chọn

Bước 3: Xử lý:

- Caesar/Vigenere: Tính toán dựa trên chỉ số bảng chữ cái.
- AES: Băm khóa bằng SHA-256 -> Tạo Nonce -> Mã hóa -> Tạo tag -> Đóng gói bằng Base64

Bước 4: Xuất kết quả: Hiển thị bản mã lên màn hình hoặc ghi vào file

CHƯƠNG 3. THIẾT KẾ CHƯƠNG TRÌNH

3.1. Mục tiêu thiết kế

Chương trình được xây dựng với mục tiêu trở thành một công cụ bảo mật đa năng, vừa phục vụ học thuật vừa có tính ứng dụng cao. Các mục tiêu cụ thể bao gồm :

- Mô phỏng đa dạng thuật toán : Triển khai từ các thuật toán cổ điển (Ceasar, Vigenere) đến hiện đại (AES) để so sánh hiệu quả
- Áp dụng kỹ thuật lập trình nâng cao : Sử dụng xử lý chuỗi phức tạp, quản lý bộ nhớ qua Context Manager và xử lý ngoại lệ (Exception Handling)
- Tối ưu hóa trải nghiệm (GUI) : Xây dựng giao diện trực quan, tách biệt hoàn toàn giữa logic xử lý và hiển thị

3.2. Kiến trúc hệ thống

Chương trình được thiết kế theo mô hình phân lớp (Layered Architecture), giúp tăng khả năng bảo trì và mở rộng mã nguồn.

3.2.1. Tầng giao diện (Presentation Layer)

Sử dụng thư viện **Tkinter**, tầng này chịu trách nhiệm tương tác trực tiếp với người dùng.

Kỹ thuật lập trình : Lập trình hướng sự kiện. Các nút bấm (tk.Button) được liên kết với các hàm xử lý thông qua tham số command.

Tầng này có thành phần gồm :

- Txt_input và txt_output sử dụng đối tượng tk.Text để xử lý các luồng văn bản lớn.

- Var_algo sử dụng tk.StringVar để quản lý trạng thái lựa chọn thuật toán của người dùng một cách động

3.2.2. Tầng điều phối và kiểm soát (Controller Layer)

Tầng này đóng vai trò trung gian, tiếp nhận dữ liệu từ GUI, kiểm tra tính hợp lệ và điều hướng đến thuật toán phù hợp. Tầng này gồm có :

- Hàm trọng tâm – validate_key() : Đây là kỹ thuật phòng vệ mã nguồn (Defensive Programming). Hàm sử dụng các phương thức như .isdigit() và .isalpha() để kiểm tra kiểu dữ liệu đầu vào.
- Logic điều hướng : Sử dụng cấu trúc rẽ nhánh if...elif...else để chọn hàm mã hóa dựa trên giá trị từ var_algo.get()

3.2.3. Tầng xử lý thuật toán (Algorithm Layer)

Đây chính là phần cốt lõi của chương trình, áp dụng nhiều kỹ thuật lập trình nâng cao :

❖ Thuật toán Ceasar (xử lý vòng lặp và ASCII)

- Mã hóa : Duyệt qua chuỗi bằng vòng lặp for. Sử dụng hàm ord() và chr() để thao tác trên bảng ASCII.
- Công thức mã hóa : $E(x) = (x + k) \pmod{26}$
- Giải mã : Áp dụng kỹ thuật **tái sử dụng hàm (Function Reuse)**. Hàm caesar_decrypt chỉ đơn giản gọi lại caesar_encrypt với khóa đối xứng là -key.

❖ Thuật toán Vigenere (Kỹ thuật Mảng vòng - Index Cycling):

Thuật toán này sử dụng biến chỉ số phụ j để duyệt qua từ khóa. Kỹ thuật nâng cao nằm ở phép toán $j \% key_len$, giúp khóa tự động quay vòng để tương ứng với độ dài của văn bản gốc

❖ Thuật toán AES (Xử lý dữ liệu nhị phân & Hashing):

Thuật toán này có khóa sử dụng `hashlib.sha256` để biến mật khẩu người dùng thành chuỗi bit chuẩn 128-bit (16 bytes)

Chế độ EAX : chế độ này kết hợp mã hóa và xác thực

- Nonce : Số ngẫu nhiên dùng một lần để chống tấn công phát lại.
- Tag : Thẻ kiểm tra tính toàn vẹn, đảm bảo dữ liệu không bị thay đổi

Base64 : Kỹ thuật chuyển đổi dữ liệu nhị phân sang chuỗi ký tự hiển thị được

3.2.4. Tầng lưu trữ và hỗ trợ (Storage và Support)

Tầng này xử lý tương tác với hệ điều hành và tệp tin. Kỹ thuật Context Manager sử dụng `with open()` để quản lý tài nguyên file. Kỹ thuật này tự động đóng file và giải phóng bộ nhớ kể cả khi có lỗi xảy ra.

Xử lý ngoại lệ (*exception handling except: messagebox.showerror("Sai khoa", "Key sai hoac file loi!")*). Kỹ thuật này giúp chương trình hoạt động bền bỉ, không bị dừng đột ngột khi người dùng nhận sai thông tin.

Hệ thống Log: kết hợp thư viện ``datetime`` để ghi lại nhật ký. Sử dụng chế độ mở file để ghi thêm dữ liệu mà không xóa nội dung cũ.

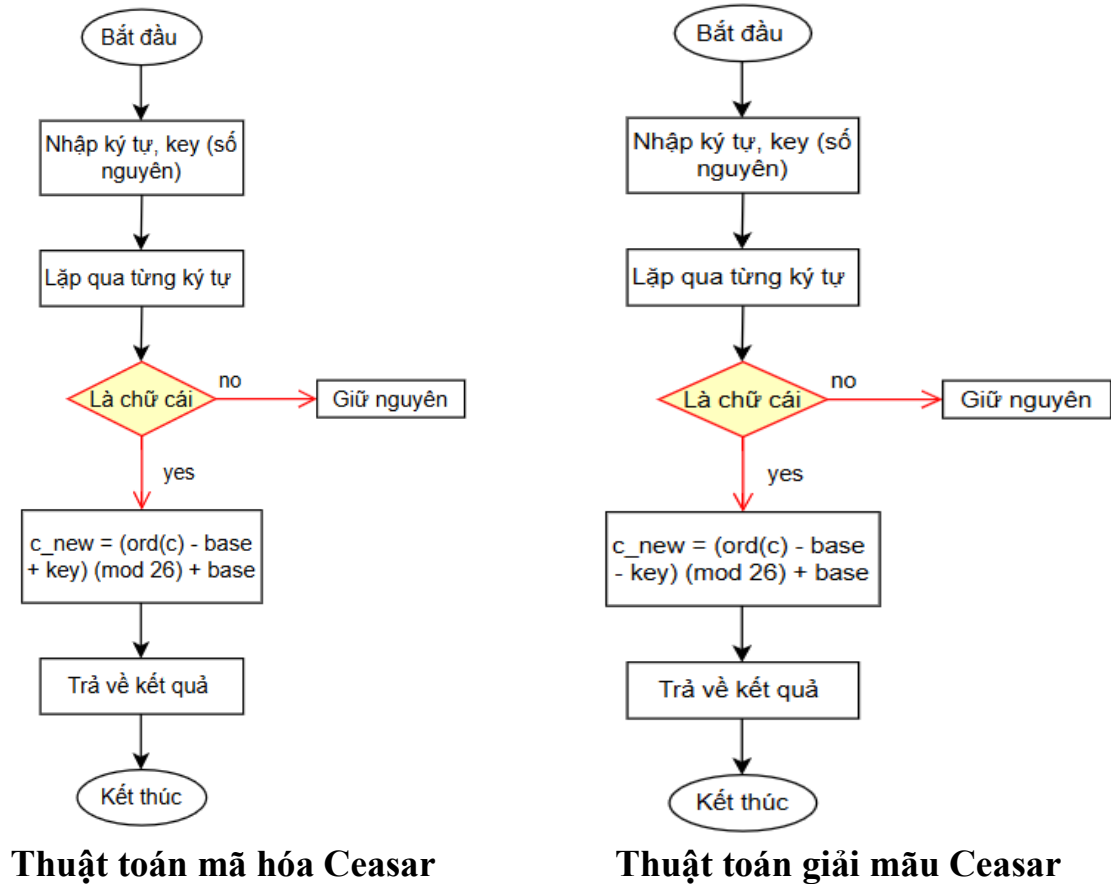
3.3. Sơ đồ khối các thuật toán chính

3.3.1. Thuật toán mã hóa Caesar

Thuật toán này đại diện cho kỹ thuật lập trình dựa trên bảng mã ASCII và toán học số dư.

Sơ đồ luồng xử lý:

1. **Bắt đầu:** Nhận chuỗi text và key (đã ép kiểu về int).
2. **Vòng lặp:** Duyệt từng ký tự c trong chuỗi.
3. **Kiểm tra:** Nếu c không phải chữ cái (số, khoảng trắng, ký tự đặc biệt) -> Giữ nguyên.
4. **Xử lý (Nếu là chữ cái):**
 - Xác định base (65 cho chữ hoa, 97 cho chữ thường).
 - Tính toán vị trí mới: $c_{\text{new}} = (\text{ord}(c) - \text{base} + \text{key}) \pmod{26} + \text{base}$.
5. **Kết thúc:** Trả về chuỗi kết quả.



Hình 3.1. Thuật toán mã hóa và giải mã Caesar

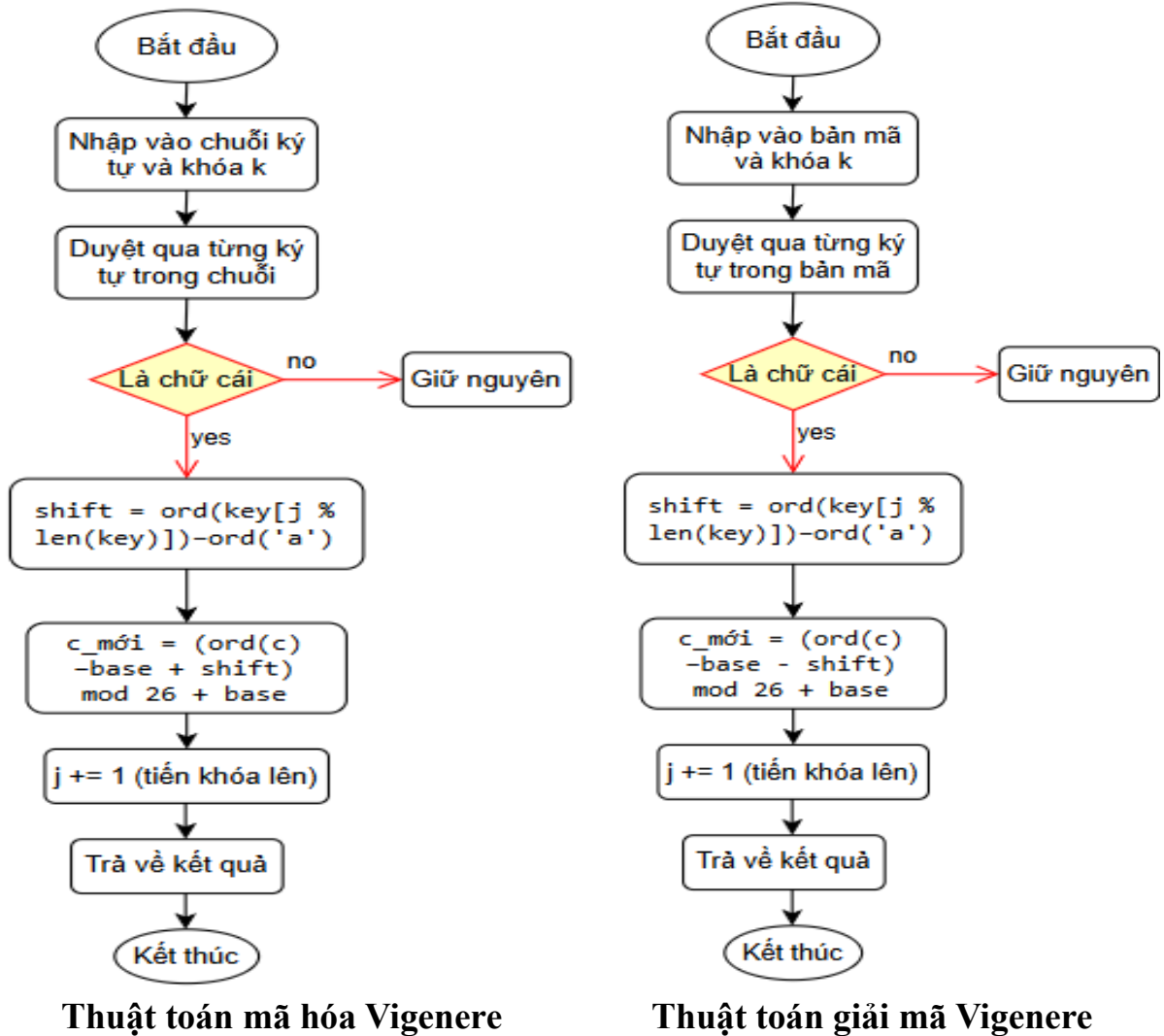
3.3.2. Thuật toán Vigenere

Điểm nâng cao ở đây là kỹ thuật **Mảng vòng (Index Cycling)** để khớp khóa có độ dài ngắn vào văn bản có độ dài lớn.

Sơ đồ luồng xử lý:

1. **Khởi tạo:** Biến $j = 0$ đóng vai trò là chỉ số chạy trên từ khóa (Key).
2. **Duyệt văn bản:** Với mỗi ký tự c , nếu là chữ cái, ta lấy bước dịch shift từ $key[j \% len(key)]$.
3. **Mã hóa:** Áp dụng phép cộng modulo tương tự Caesar nhưng bước dịch thay đổi theo từng vị trí.

4. **Cập nhật:** Chỉ tăng j khi ký tự hiện tại là chữ cái để đảm bảo tính liên tục của từ khóa.



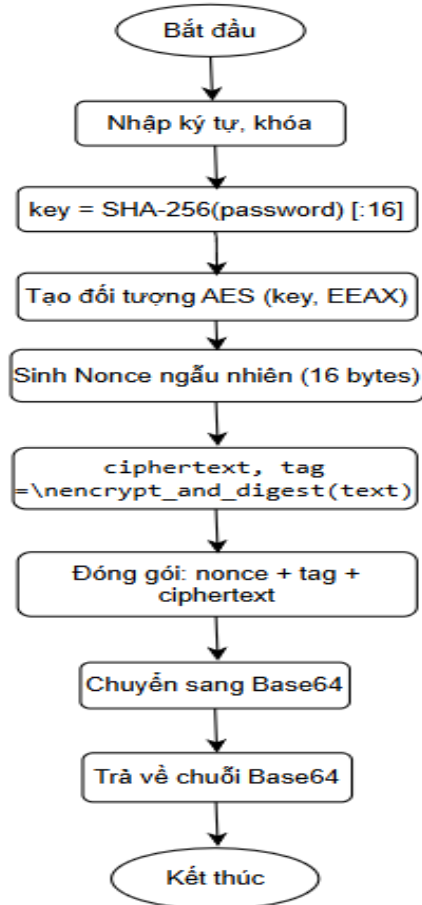
Hình 3.2. Thuật toán mã hóa và giải mã Vigenere

3.3.3. Thuật toán AES (Advanced Encryption Standard)

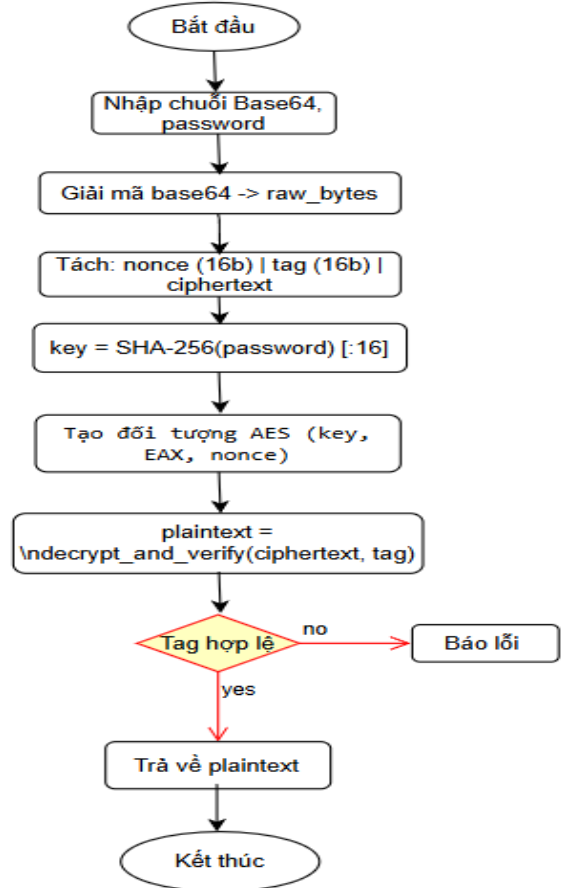
Sử dụng chế độ **EAX** – một kỹ thuật lập trình bảo mật hiện đại giúp đảm bảo cả tính bảo mật và tính toàn vẹn dữ liệu.

Sơ đồ luồng xử lý:

1. **Băm khóa:** Sử dụng SHA-256 để biến mật khẩu bất kỳ thành một chuỗi 16-byte chuẩn.
2. **Khởi tạo:** Tạo đối tượng AES với khóa vừa băm và chế độ MODE_EAX.
3. **Mã hóa:** Hàm `encrypt_and_digest` trả về ciphertext và thẻ xác thực tag.
4. **Đóng gói:** Kết hợp nonce (số dùng một lần) + tag + ciphertext thành một khối nhị phân.
5. **Base64:** Chuyển khối nhị phân sang dạng văn bản để hiển thị.



Thuật toán mã hóa AES

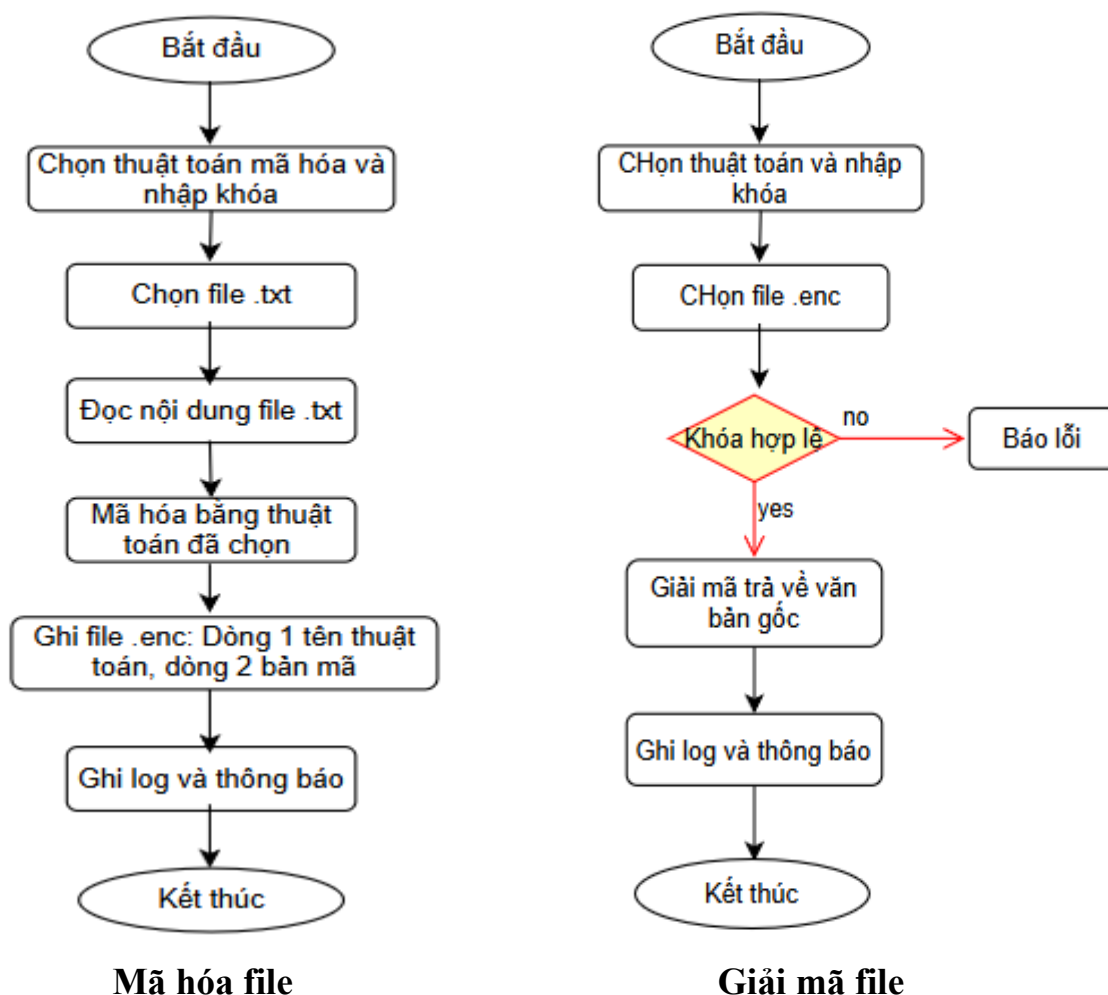


Thuật toán giải mã AES

Hình 3.3. Thuật toán mã hóa và giải mã AES

3.3.4. Quy trình mã hóa và giải mã file

Chương trình thiết kế một cấu trúc file có **Header** để tăng tính tự động hóa (Automation).



Hình 3.4. Mã hóa và giải mã file

3.4. Cấu trúc dữ liệu

Chương trình sử dụng linh hoạt các cấu trúc dữ liệu để tối ưu hóa hiệu năng:

- **Chuỗi ký tự (String):** Dùng để lưu trữ văn bản gốc và bản mã. Python xử lý String dưới dạng Unicode, giúp chương trình hỗ trợ tốt tiếng Việt.

- **Số nguyên (Integer):** Dùng làm khóa cho Caesar và các biến chỉ số (index) trong vòng lặp.
- **Kiểu Byte (Bytes):** Đây là cấu trúc quan trọng nhất trong AES. Mọi dữ liệu trước khi mã hóa AES đều được chuyển về bytes qua phương thức `.encode()`.
- **Danh sách (List):** Sử dụng hàm `readlines()` để đọc file theo dòng, giúp tách Header thuật toán ra khỏi dữ liệu mã hóa dễ dàng.
- **Biến trạng thái GUI:** `tk.StringVar()` giúp đồng bộ hóa lựa chọn thuật toán trên giao diện với logic xử lý bên dưới.

3.5. Các hàm chính trong chương trình

3.5.1. Hàm kiểm tra khóa `validate_key()`

```
def validate_key():
    k = entry_key.get().strip()
    algo = var_algo.get()

    if algo == "Caesar":
        if not k.isdigit():
            messagebox.showerror("Loi", "Key Caesar phai la so nguyen")
            return None
        return int(k)

    elif algo == "Vigenere":
        if not k.isalpha():
            messagebox.showerror("Loi", "Key Vigenere chi duoc chua chu cai")
            return None
        return k.upper()

    elif algo == "AES":
        if k == "":
            messagebox.showerror("Loi", "Key AES khong duoc rong")
            return None
        return k # se encode thanh byte sau

    return None
```

3.5.2. Hàm mã hóa và giải mã của mã hóa Caesar

```
def caesar_encrypt(text, key):
```

```

kq = ""
for c in text:
    if c.isalpha():
        shift = key % 26
        if c.isupper():
            kq += chr((ord(c)-65+shift)%26+65)
        else:
            kq += chr((ord(c)-97+shift)%26+97)
    else:
        kq += c
return kq

def caesar_decrypt(text, key):
    return caesar_encrypt(text, -key)

```

3.5.3. Hàm mã hóa và giải mã của mã hóa Vigenere

```

def vigenere_encrypt(text, key):
    kq=""
    key_len=len(key)
    j=0

    for c in text:
        if c.isalpha():
            shift=ord(key[j%key_len].lower())-97
            if c.isupper():
                kq+=chr((ord(c)-65+shift)%26+65)
            else:
                kq+=chr((ord(c)-97+shift)%26+97)
            j+=1
        else:
            kq+=c
    return kq

def vigenere_decrypt(text, key):
    kq=""
    key_len=len(key)
    j=0

    for c in text:
        if c.isalpha():
            shift=ord(key[j%key_len].lower())-97
            if c.isupper():
                kq+=chr((ord(c)-65-shift)%26+65)
            else:
                kq+=chr((ord(c)-97-shift)%26+97)
            j+=1
        else:

```

```
kq+=c
return kq
```

3.5.4. Hàm mã hóa và giải mã của mã hóa AES

```
def aes_encrypt(text, key):
    key = hashlib.sha256(key.encode()).digest()[:16]
    cipher = AES.new(key, AES.MODE_EAX)
    nonce = cipher.nonce
    ciphertext, tag = cipher.encrypt_and_digest(text.encode())
    return base64.b64encode(nonce + tag + ciphertext).decode()

def aes_decrypt(text, key):
    key = hashlib.sha256(key.encode()).digest()[:16]
    data = base64.b64decode(text)
    nonce = data[:16]
    tag = data[16:32]
    ciphertext = data[32:]

    cipher = AES.new(key, AES.MODE_EAX, nonce=nonce)
    plaintext = cipher.decrypt(ciphertext)
    cipher.verify(tag)
    return plaintext.decode()
```

3.6. Phân tích dữ liệu đầu vào

Hệ thống xử lý hai loại dữ liệu chính để đảm bảo tính nhất quán và ngăn ngừa lỗi runtime:

3.6.1. Chuỗi văn bản (Plaintext/Ciphertext)

- Kiểu dữ liệu string (UTF-8)
- Đặc điểm là chấp nhận mọi ký tự Unicode bao gồm chữ cái, số, ký tự đặc biệt và khoảng trắng

3.6.2. Khóa (Secret Key)

Mỗi thuật toán yêu cầu một định dạng khóa riêng biệt để đảm bảo tính đúng đắn của phép toán.

- Thuật toán Ceasar khóa có kiểu dữ liệu là số nguyên để thực hiện phép cộng chỉ số.
- Thuật toán Vigenere khóa có kiểu dữ liệu là kiểu string, chỉ chứa ký tự chữ cái (A-Z, a-z)
- Thuật toán AES khóa có kiểu dữ liệu là string, là chuỗi ký tự bất kỳ vì sẽ được băm thành độ dài chuẩn

3.7. Thiết kế hàm kiểm tra khóa (Key validation)

Mục tiêu của hàm này là thực hiện **Defensive Programming** (Lập trình phòng vệ) để tránh lỗi ép kiểu dữ liệu.

Cấu trúc logic:

- Nếu chọn Ceasar: Sử dụng `k.isdigit()` để đảm bảo người dùng nhập số.
- Nếu chọn Vigenere: Sử dụng `k.isalpha()` để đảm bảo khóa không chứa số hay ký tự lạ.
- Nếu chọn AES: Kiểm tra khóa không được để trống.

Kỹ thuật phản hồi ngay lập tức qua `messagebox.showerror` để chặn luồng xử lý lỗi từ đầu.

3.8. Thiết kế xử lý file

Luồng xử lý của file sẽ được thực hiện như sau:

- Mã hóa: Đọc file nguồn -> Mã hóa dữ liệu -> Ghi Header + Data vào file mới.
- Giải mã: Đọc Header để chọn thuật toán -> Đọc Data -> Giải mã -> Ghi trả lại file gốc.

Kỹ thuật sử dụng Context Manager (with open) để quản lý file an toàn và tránh rò rỉ tài nguyên hệ thống.

3.9. Thiết kế GUI (giao diện người dùng)

- **Thành phần chính:** Sử dụng các Widget của tkinter như Text, Entry, OptionMenu, và Button.
- **Cơ chế Event-driven:** Chương trình chạy trong một vòng lặp sự kiện (mainloop). Khi người dùng tương tác (click nút), một **Callback function** (như ma_hoa, giai_ma) sẽ được kích hoạt để thực thi logic tương ứng.

3.10. Thiết kế ghi Log (nhật ký hệ thống)

- **Mục tiêu:** Ghi lại lịch sử hoạt động để hỗ trợ Debug và Audit (Kiểm toán an ninh).
- **Định dạng:** [Thời gian thực] - [Hành động] - [Thông tin chi tiết].
- **Kỹ thuật:** Mở file ở chế độ **Append mode** ("a") để bảo toàn dữ liệu cũ và cập nhật dữ liệu mới một cách liên tục.

CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

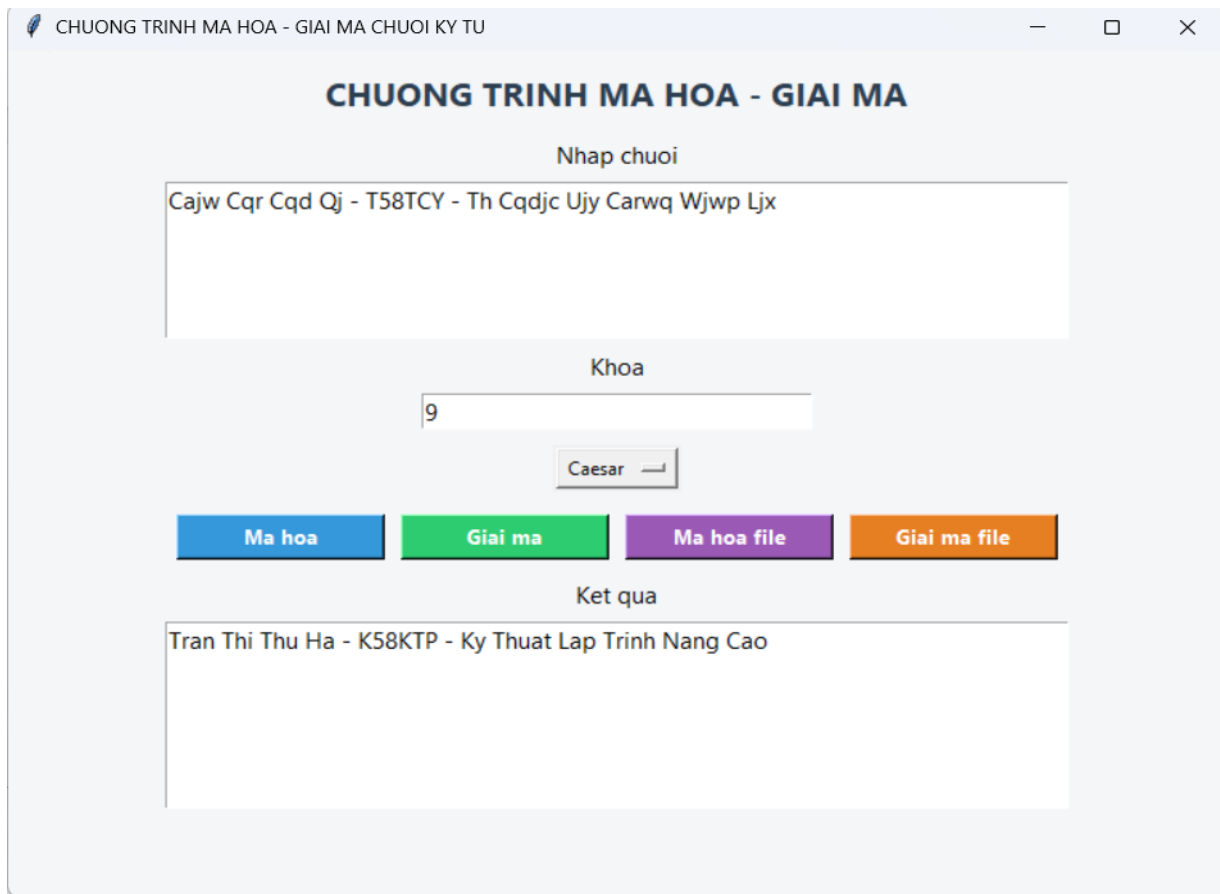
4.1. Kết quả kiểm thử

Trong quá trình làm đề tài thiết kế chương trình mã hóa – giải mã chuỗi ký tự, em đã tiến hành kiểm thử toàn bộ các chức năng chính của hệ thống như mã hóa và giải mã chuỗi ký tự bằng thuật toán mã hóa Ceasar, Vigenere, AES, thực hiện mã hóa và giải mã file ký tự. Tất cả các tính năng chính này đều hoạt động ổn định và cho ra kết quả đúng như mong đợi, giao diện người dùng thân thiện, dễ sử dụng và hiển thị thông tin rõ ràng, không phát hiện lỗi trong quá trình kiểm thử. Dưới đây là kết quả kiểm thử của 1 thuật toán sau khi chạy chương trình.

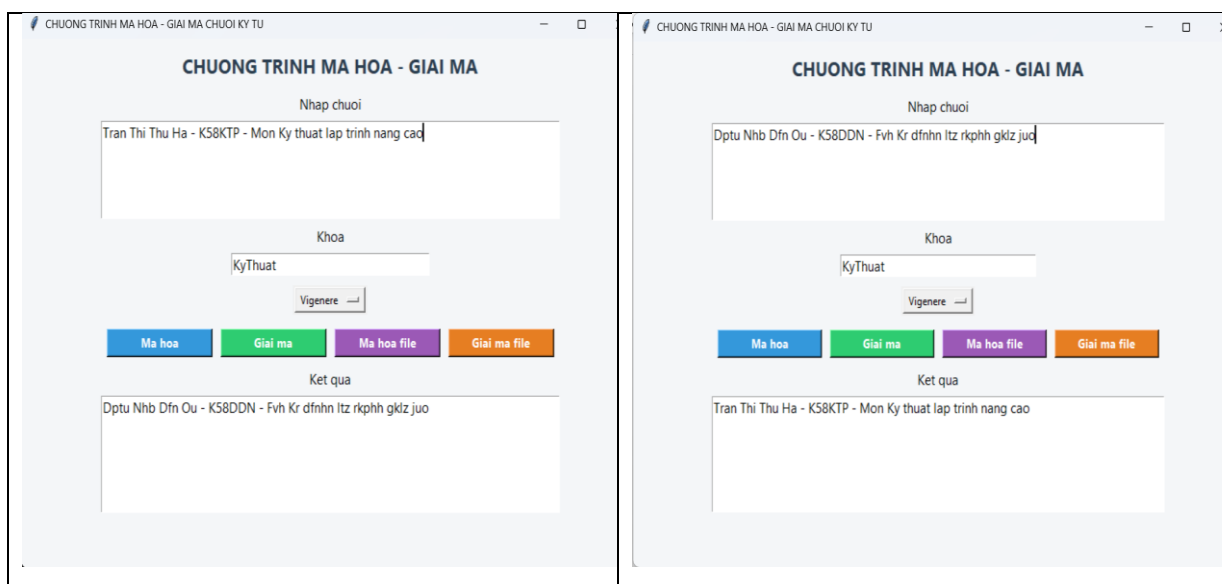


The screenshot shows a web application interface for testing encryption and decryption. The title bar reads 'CHƯƠNG TRÌNH MÃ HOA - GIẢI MÃ CHUOI KY TU'. The main heading is 'CHƯƠNG TRÌNH MÃ HOA - GIẢI MÃ'. Below it, there is a section for input: 'Nhap chuoi' with a text box containing 'Tran Thi Thu Ha - K58KTP - Ky Thuat Lap Trinh Nang Cao'. Another input field 'Khoa' contains the number '9'. A dropdown menu is set to 'Caesar'. There are four buttons: 'Ma hoa' (blue), 'Giải ma' (green), 'Ma hoa file' (purple), and 'Giải ma file' (orange). Below these is a section for output: 'Ket qua' with a text box displaying the encrypted result: 'Cajw Cqr Cqd Qj - T58TCY - Th Cqđjc Ujy Carwq Wjwp Ljx'.

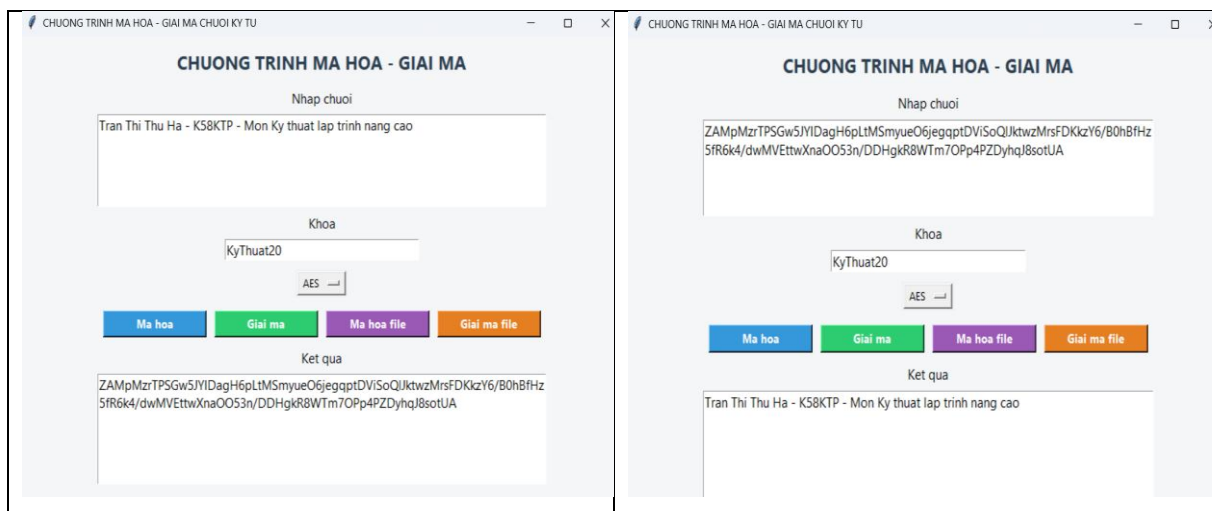
Hình 4.1. Kết quả kiểm thử mã hóa Ceasar



Hình 4.2. Kết quả kiểm thử thuật toán giải mã Ceasar



Hình 4.3. Kết quả kiểm thử thuật toán Vigenere



Hình 4.4. Kết quả kiểm thử thuật toán AES

4.2. Kết luận

4.2.1. Kết quả sản phẩm đạt được

Sau quá trình tìm hiểu và triển khai, chương trình mã hóa và giải mã chuỗi ký tự đã được xây dựng thành công với các chức năng cơ bản là mã hóa và giải mã chuỗi ký tự, mã hóa và giải mã được file ký tự, có ghi log. Giao diện người dùng thân thiện và trực quan với các thành phần như button, Treeview giúp người dùng thao tác dễ dàng và hiệu quả. Dữ liệu được lưu trữ và đồng bộ hóa kịp thời sau mỗi lần hành động của người dùng. Hệ thống cũng xử lý tốt các lỗi đầu vào, lỗi định dạng và đảm bảo an toàn dữ liệu trong quá trình sử dụng.

4.2.2. Kiến thức đã học được trong quá trình làm đề tài

Trong quá trình thực hiện đề tài này em đã củng cố và nâng cao kiến thức về lập trình python, đặc biệt là cách làm việc với chuỗi ký tự ở tầng mã ASCII thông qua các hàm `ord()` và `chr()`, xử lý dữ liệu nhị phân với kiểu bytes, cũng như ứng dụng các thư viện chuyên dụng như PyCryptodome và hashlib vào bài toán thực tế. Bên cạnh đó em cũng hiểu sâu hơn về các kỹ thuật lập trình nâng cao như

tái sử dụng hàm (function reuse), quản lý tài nguyên file bằng Context Manager và xử lý ngoại lệ (Exception handing) để chương trình hoạt động ổn định.

Ngoài ra em còn học được cách xây dựng giao diện người dùng GUI bằng thư viện Tkinter, tiếp cận mô hình lập trình hướng sự kiện và quản lý trạng thái giao diện thông qua StringVar. Việc tổ chức mã nguồn theo kiến trúc phân lớp (layer Architecture) tách biệt rõ tầng giao diện, điều phối và xử lý thuật toán cũng giúp em hiểu được tầm quan trọng của nguyên tắc Separation of Concerns trong thiết kế phần mềm thực tế.

Bên cạnh đó kỹ năng tư duy thuật toán và thiết kế cấu trúc dữ liệu cũng được cải thiện rõ rệt thông qua việc nghiên cứu và triển khai 3 thuật toán có độ phức tạp tăng dần từ Ceasar, Vigenere đến AES. Quá trình kiểm thử nhiều trường hợp khác nhau – gồm cả các trường hợp biên và trường hợp lỗi còn rèn luyện thêm thói quen tư duy cẩn thận và có hệ thống khi viết code.

Những kiến thức và kinh nghiệm đã tích lũy được từ đề tài này không chỉ giúp em hoàn thiện kỹ năng lập trình hiện tại mà còn tạo nền tảng vững chắc để tiếp tục học hỏi các chủ đề nâng cao hơn trong lĩnh vực bảo mật phần mềm và phát triển ứng dụng thực tế ở các môn học tiếp theo.

TÀI LIỆU THAM KHẢO

1. https://en.wikipedia.org/wiki/Vigen%C3%A8re_cipher
2. Phan Đình Diệu. *Lý thuyết mật mã và an toàn thông tin*. NXB Đại học Quốc gia Hà Nội, 2002.
3. Trần Nhật Quang, Phạm Văn Khoa. *Giáo trình Lập trình Python căn bản*. Trường Đại học Công nghiệp TP. Hồ Chí Minh.
4. Thái Thanh Tùng. *Giáo trình Mật mã học và Hệ thống thông tin an toàn*. Viện Đại học Mở Hà Nội, 2006.

LINK GITHUB

1. link : https://github.com/ThuHa2004/BTL_KTLTNC.git

2. QR github :

